

SEMINAR: EF CORE TRACKING VS NOTRACKING

Hiểu ChangeTracker để tối ưu hiệu năng thực tế

I. GIỚI THIỆU VẤN ĐỀ (MỞ ĐẦU)

Trong nhiều hệ thống thực tế:

- API list dữ liệu chậm khi record tăng
- Memory server tăng dần
- CPU spike dù query không phức tạp

Câu hỏi:

Có phải SQL Server chậm?
Hay network chậm?
Hay EF Core chậm?

Câu trả lời thường nằm ở:

ChangeTracker của EF Core

II. NỀN TẢNG LÝ THUYẾT

1 ORM là gì?

ORM (Object Relational Mapper) giúp:

- Map table → class
- Map row → object
- Map column → property

Trong .NET, EF Core là ORM phổ biến nhất.

2 Unit of Work Pattern

EF Core hoạt động theo mô hình:

Load → Modify → SaveChanges → Commit

DbContext chính là Unit of Work.

3 ChangeTracker là gì?

ChangeTracker là thành phần:

- Theo dõi mọi entity được load
- Lưu trạng thái entity:
 - Unchanged
 - Modified
 - Added
 - Deleted
- Lưu snapshot giá trị ban đầu

Cấu trúc nội bộ:

Dictionary<EntityKey, EntityEntry>

4 Tracking hoạt động như thế nào?

Trong .NET, tracking và non-tracking queries là những cách để tương tác với cơ sở dữ liệu.

Trong lập trình .NET, tracking là tính năng của Entity Framework Core (EF Core) để theo dõi trạng thái của các đối tượng trong cơ sở dữ liệu. Khi sử dụng tracking, EF Core sẽ theo dõi và lưu trữ các thay đổi của đối tượng trong bộ đệm thay đổi. Khi gọi phương thức SaveChanges(), các thay đổi này sẽ được lưu vào cơ sở dữ liệu.

Tracking được kích hoạt mặc định cho tất cả các truy vấn trong EF Core. Tuy nhiên, trong một số trường hợp, sử dụng tracking có thể làm giảm hiệu suất và tốn tài

nguyên. Vì vậy, khi chỉ đọc dữ liệu từ cơ sở dữ liệu mà không cần lưu trữ thay đổi, ta nên sử dụng non-tracking query để cải thiện hiệu suất

Khi bạn query:

```
var orders = db.Orders.ToList();
```

EF sẽ:

1. Tạo object Order
2. Tạo EntityEntry
3. Lưu snapshot original values
4. Thêm vào ChangeTracker

5 Detectchange hoạt động ra sao?

Khi SaveChanges():

- EF so sánh snapshot và current values
- Nếu khác → đánh dấu Modified

Độ phức tạp gần $O(n \times m)$:

- n = số entity
- m = số property

III. TRACKING VS NOTRACKING – KHÁC NHAU Ở ĐÂU?

Tracking (mặc định)

Ưu điểm:

- Update tự động

- Navigation fix-up
- Identity resolution
- Phù hợp CRUD

Nhược điểm:

- Tốn memory
- Tốn CPU
- Không cần thiết cho read-only

NoTracking

.AsNoTracking()

EF sẽ:

- Không tạo EntityEntry
- Không snapshot
- Không DetectChanges
- Chỉ load rồi trả về

Phù hợp:

- API list
- Report
- Dashboard
- Read-heavy microservice

IV. GIỚI THIỆU PROJECT DEMO

Project gồm:

- AddDbContext
- Entities: Order, Customer, OrderItem, Product
- Endpoint tracking
- Endpoint notracking
- Perf wrapper đo:
 - elapsed
 - trackedEntries
 - memory Delta Bytes

V. DEMO TRÊN SWAGGER

Bước 1 – Test Tracking

Swagger:

GET /api/orders/tracking

Set:

take = 100

Execute.

Kéo xuống phần meta.

Ví dụ kết quả:

trackedEntries: 800
memory Delta Bytes: 2 MB
elapsed Ms: 1200

Giải thích:

- 100 Orders
 - Items
 - Product

- Customer
→ Tổng ~800 entity được tracking

Bước 2 – Tăng take lên 300

Set:

take = 300

Ví dụ kết quả:

trackedEntries: 2600+
memory Delta Bytes: 5 MB
elapsed Ms: 5600 ms

Giải thích:

Tracking tăng tuyến tính theo dataset.

Bước 3 – Test NoTracking với take = 300

GET /api/orders/nottracking?take=300

Ví dụ:

trackedEntries: 0
memory Delta Bytes: ~1 MB
elapsed Ms: ~400 ms

MỤC TIÊU TEST 2

Test 1 chỉ so sánh 1 giá trị take.

Test 2 trả lời câu hỏi lớn hơn:

Khi dataset tăng dần thì Tracking và NoTracking scale như thế nào?

BƯỚC 1 – Chạy endpoint

Vào Swagger:

GET /api/test/compare-scaling

Không cần tham số.

Execute.

Bạn sẽ nhận được dạng:

```
[
  {
    "take": 100,
    "tracking Time Ms": 600,
    "trackingEntries": 900,
    "noTracking Time Ms": 70
  },
  {
    "take": 300,
    "tracking Time Ms": 1800,
    "trackingEntries": 2700,
    "noTracking Time Ms": 120
  },
  {
    "take": 600,
    "tracking Time Ms": 4200,
    "trackingEntries": 5500,
    "noTracking Time Ms": 210
  }
]
```

(Số của bạn sẽ khác nhưng xu hướng giống)

PHÂN TÍCH CHI TIẾT

1 Tracking Entries tăng tuyến tính

Ví dụ:

take	trackedEntries
------	----------------

100	~900
-----	------

300	~2700
-----	-------

600	~5500
-----	-------

Gần như tăng tuyến tính theo take.

Vì:

- Mỗi Order kéo theo
 - 1 Customer
 - N Items
 - N Product

Tracking phải giữ toàn bộ graph.

NoTracking Time tăng chậm hơn

Ví dụ:

take	noTracking Time
100	70 ms
300	120 ms
600	210 ms

Tăng nhưng không tăng đột biến.

Vì:

- Chỉ materialize object
- Không lưu state
- Không snapshot
- Không detect change

VI. SO SÁNH TRỰC TIẾP

Metric	Tracking	NoTracking
trackedEntries	2600+	0
memory	5MB	1MB

elapsed 5.6s 0.38s

Tracking chậm hơn ~10–15 lần.

VII. PHÂN TÍCH KỸ THUẬT

Tracking tốn chi phí ở:

1. Allocation EntityEntry
2. Snapshot original values
3. Navigation fix-up
4. DetectChanges
5. Identity map maintenance

NoTracking:

- Bỏ toàn bộ các bước trên
- Chỉ materialize object

VIII. KHI NÀO DÙNG TRACKING?

- CRUD truyền thống
- Update entity
- Domain-driven design
- Business logic phức tạp

IX. KHI NÀO DÙNG NOTRACKING?

- API list
- Report

- Dashboard
- Read-heavy service
- CQRS query side

X. BEST PRACTICES

1. Read-heavy → Global NoTracking
2. Update đơn giản → ExecuteUpdate
3. Không trả entity trực tiếp → dùng DTO
4. DbContext lifetime → Scoped

XI. KẾT LUẬN

EF Core không chậm.

ChangeTracker là công cụ mạnh,
nhưng phải dùng đúng chỗ.

Tracking không sai.

NoTracking không tốt hơn tuyệt đối.